

Security, integrity and performance audit

An audit of the Questbase Command Center carried out against live production, the live Supabase database and the current build. Twenty-two findings are recorded below: fourteen were remediated during the audit, three are accepted risks with stated reasoning, and five remain open and require action outside engineering.

REPORT DATE	1 August 2026
SYSTEM	Questbase Command Center – www.questbase.io
REVISION AUDITED	8766382 (main)
BASELINE	972dffa4 – 19 commits earlier
DATABASE	Supabase rqundirizvojzphljtdn • PostgreSQL 17.6
TEST BASELINE	902 of 902 passing
PREPARED BY	Claude Opus 5, working session audit

Scope and method

Findings were produced by four methods: reading the application and migration source; querying the live database catalogue directly; running the Supabase security and performance advisors and *classifying* rather than counting their output; and executing probes against live production inside transactions that were rolled back.

Rollback-only probing matters to how much weight these findings carry. Where a finding says a policy now rejects something, that was demonstrated by attempting the action as a real member identity and observing the refusal — not inferred from reading the policy. No production data was created or modified by this audit; each probe was verified afterwards to have left no trace.

Limitations — what this audit does not cover

- No penetration testing, fuzzing, or authenticated session-hijacking analysis was performed.
- No two-identity browser rehearsal. Tenancy separation is evidenced at the database layer only; the owner-plus-restricted-worker walkthrough remains outstanding and is recorded as F-19.

- No email deliverability testing. Auth, invite and recovery mail paths exist in code but reaching a monitored inbox has not been proven (F-20).
- Accessibility covers programmatic names, keyboard focus behaviour and colour contrast. It does not cover screen-reader walkthroughs or a device viewport matrix (F-18).
- Third-party dependency and supply-chain review was out of scope.

Summary of findings

SEVERITY	REMEDIATED	ACCEPTED	OPEN	TOTAL
High	2	0	0	2
Medium	8	0	3	11
Low	4	1	2	7
Informational	0	2	0	2
Total	14	3	5	22

Both high-severity findings were tenancy or data-integrity defects that existing tests did not detect, and neither appeared on the project's own release checklist. Both were found by reading code closely rather than by any automated signal, which is the principal recommendation of this audit: advisor output should be classified, not counted.

Access control and data isolation

F-01 • ROW-LEVEL SECURITY

HIGH

REMIEDIATED

Three messaging policies contained checks that were always true

Three row-level security policies on `messages`, `message_attachments` and `message_conversation_access` compared a column to itself — for example `mc.company_id = mc.company_id`. Such a comparison is a tautology and enforces nothing.

The clauses were plainly intended to bind a new row to its parent: a message's company must match its conversation's, and an attachment's conversation and company must match the message it belongs to. As written, a client could insert a message carrying one company's identifier into a conversation belonging to another company. Only a well-behaved client prevented it.

Before tightening, all existing rows were checked against the intended constraint across four relationships; none violated it. The remediation therefore closes an exposure without invalidating data in use.

Evidence — acting as a real member identity, rolled back: an ordinary send was accepted, a message tagged with a foreign company identifier was rejected, and a message sent as another user was rejected. Migration `202608010900`.

F-02 • API SURFACE

MEDIUM

REMIEDIATED

A trigger function was callable by anonymous users over the REST API

`touch_eod_report_updated_at()` is a trigger function, invoked by the table it is attached to. PostgreSQL grants EXECUTE on new functions to PUBLIC by default, and PostgREST exposes anything the browser roles can execute as an RPC endpoint. The function was therefore reachable at `/rest/v1/rpc/` without signing in.

Impact is limited — the function only stamps a timestamp — but the exposure class is not, and this is a defect pattern rather than an isolated mistake: any future `returns trigger` function ships the same way unless its grant is explicitly revoked.

Evidence — of six SECURITY DEFINER trigger functions in `public`, five already carried no browser grants; this one carried PUBLIC, anon and authenticated. EXECUTE revoked from all three; trigger firing confirmed unaffected afterwards. A regression test now derives every trigger function from the migration history and fails if one ships unrevoked.

F-03 • TABLE GRANTS

LOW

REMIEDIATED

Two server-only tables carried browser grants that implied access they did not give

`checkin_log` and `reminder_log` have row-level security enabled with no policies, which denies all access to non-owner roles, yet both still granted table privileges to anon and authenticated. Not an exposure in itself, but the grants described a path that did not exist and would have become real the moment anyone added a policy.

Evidence — grants revoked; both tables now read unambiguously as server-only in the schema.

47 SECURITY DEFINER functions are executable by authenticated users

The security advisor reports these as warnings. On review they are the application's intended API — invite acceptance, role management, the recycle-bin and workspace operations — and each performs its own permission checks with a fixed `search_path`. Accepted as designed. They are listed separately from F-02 because the distinction that matters is trigger function versus RPC, which the advisor does not draw.

Leaked-password protection is disabled

Supabase can reject passwords known to have appeared in public breach corpora. The setting is off. Application-side password rules exist but do not cover this. Enabling it is a dashboard toggle and requires an account owner.

Owner action — Supabase dashboard, Authentication settings. Confirmed still disabled at time of writing.

Data integrity

Concurrent App Builder edits silently destroyed each other

A company's entire App Builder document is stored as a single JSONB row, and saving performed an unconditional upsert. Two people editing at once meant the second save overwrote the first completely — no error, no warning, and the loss was discoverable only by noticing that work had vanished.

Saves are now conditional on the revision they were based on, using the row's existing trigger-maintained `updated_at` as a version token, so no schema change was required. A save built on a stale read matches zero rows instead of overwriting; it then performs a three-way merge against the last-synced revision and retries. Because every entity in the document carries a stable identifier, edits to different apps, records or fields reconcile without user involvement.

Evidence — against production, rolled back: two clients loading the same revision, the first write accepted, the second stale write matched zero rows, and the retry against the new revision accepted. Sixteen tests cover the merge rules directly.

A second copy of the document-apply logic bypassed the new concurrency controls

The realtime refresh path held its own duplicate of the logic that applies builder rows to memory, and that copy did not update the version token or merge ancestor introduced by F-06. After any realtime refresh the client's bookkeeping described a revision that was no longer current, so the next save would fail its guard and then merge against a stale ancestor — precisely the failure the guard exists to prevent.

This was introduced by the F-06 remediation earlier the same day and found while auditing an unrelated area. Duplicated logic was the cause; there is now one implementation with a test asserting a single definition and two call sites.

Apps shared between workspaces were treated as absent by the interface

Installing an app into a second workspace stores a pointer, so both workspaces operate on one app with shared fields and records. The data model was correct, but three interface paths filtered on "not linked" and so ignored such apps entirely. Most visibly, a workspace whose only app had been installed from elsewhere was given no app tile at all, making a correctly functioning feature appear not to work.

Root cause – an application identifier is identical for a linked entry and its source, so testing for "not linked" cannot distinguish "a different app" from "reach this one through the link". All three paths now resolve the link. Ten tests added, including mutation through the link in both directions.

Pre-existing CRM handoff records may contain duplicates and missing links

Contact-to-quote conversion is now atomic and idempotent, so new records cannot develop these problems. Records created before that landed still can. A review screen has been built that reports duplicates and incomplete links and links directly to each record.

The screen deliberately offers no bulk-repair action. A mass rewrite of pilot data cannot be undone and the correct resolution differs per record, so this requires a person working through the list.

Owner action – Settings › Data review, visible to roles holding CRM management permission.

Performance and scalability

F-10 • QUERY PLANNING

MEDIUM

REMIEDIATED

Seventeen policies re-evaluated the authentication function for every row

Each called `auth.uid()` or `auth.jwt()` directly, causing evaluation once per row rather than once per query. Wrapping each call in a scalar subquery allows the planner to hoist it into a single initialisation step. The value is identical; only the number of calls changes. The effect is largest on the messaging tables, which are queried on every poll of the inbox.

Verification – rewrites were checked mechanically rather than by eye: normalising the new definitions back to their original form leaves no differences beyond the intended ones, and no policy in `public` or `app_private` retains an unhoisted call.

F-11 • INDEXING

MEDIUM

REMIEDIATED

Seven foreign keys had no covering index

An unindexed foreign key turns deletion of a parent row into a full scan of the child table while holding locks. Deleting a role or a profile would have been the first administrative action to start timing out as data grew.

One index required a second attempt for a reason worth recording: `create index if not exists` matches on the index *name*, and an index already existed under the intended name on a composite whose leading column was different — so it silently did nothing while the key remained uncovered. A migration reporting success is not evidence that the index exists.

Evidence – the uncovered-foreign-key query now returns none.

F-12 • STARTUP COST

LOW

REMIEDIATED

Fifty-one tables were fetched before the first screen could render

Eight domains — finance, forms, pricebook, client portals, recycle bin, audit, underwriting and proposals — now load when first requested rather than at startup, reducing this to thirty-four. Measurement preceded the change and corrected the assumption behind it: payload is not the constraint, since the largest table holds 338 rows. The cost is round trips, and the durable benefit is that these do not grow with the business.

Safeguard – deferral is triggered by the data accessors rather than by route, so no screen can read a dataset without causing it to load. An automated check enumerates every read and fails if one is reached without a trigger; it identified six such readers during implementation.

The client bundle has 141 bytes of headroom against its budget

This is now the binding constraint on further development: the next feature of any size fails the build check. The budget has already been raised four times, so raising it again warrants a deliberate decision rather than a reflex.

Three remedies were measured. The instructive result is negative: the fifty self-contained helper functions in the App Builder — twenty-one kilobytes, and an apparently ideal extraction — cannot help, because they are also called from paths that run at startup and so must remain in the main bundle. Only deferring the App Builder editing interface reduces the compressed size, and that requires separating 220 declarations spread across 4,477 lines into shared and deferred modules.

Recommendation — schedule this as discrete work with the ability to exercise the App Builder afterwards. An automated suite cannot detect that a configuration panel has silently stopped rendering.

Fifteen tables carry multiple permissive policies for the same action

The advisor recommends merging them into single compound policies for planner efficiency. Declined. These policies constitute the tenancy boundary — the property this system has invested most effort in proving correct — and one readable policy per intent is what keeps that boundary auditable. A merged disjunction trades a minor planning gain for exactly the kind of expression in which a tenancy error would hide. Revisit if profiling shows it matters.

133 indexes have not been used

Expected for a database at this stage: most tables hold tens of rows, so the planner correctly prefers sequential scans and index usage statistics remain near zero. Not actionable until production data volumes are representative; removing indexes on this evidence would be premature.

Accessibility

Dialogs neither received nor returned keyboard focus

Opening a dialog left focus on the control that opened it — which the application destroys when it re-renders — so focus fell to the document body. A screen reader therefore never announced the dialog, and closing one returned the user to the top of the page rather than to their place. Both defects are invisible to anyone using a mouse.

Focus now moves into the dialog on open and returns to the originating control on close. Because the original element does not survive re-rendering, the return target is recorded as a selector rather than a reference.

Also corrected — the App Builder dialog was not identified as a dialog to assistive technology; the general dialog had no accessible name; five icon-only controls had no name. The icon-picker grids were the most severe, every option announcing identically as "button".

Secondary text fell below WCAG AA in six places

Four interface themes and two colours on the public marketing page measured between 3.96:1 and 4.45:1 against the surfaces they appear on, below the 4.5:1 required for normal text. All now exceed it, changed by the smallest amount that clears the threshold so the visual design is preserved.

Method note – a first pass examined only the first definition of each colour and reported a single failure. The stylesheet layers several theme blocks, so contrast must be computed per theme against that theme's own background; scoping it correctly turned one finding into four. Two further failures were on the marketing page, which carries an entirely separate palette. Ratios are now recomputed automatically from the stylesheet.

Viewport matrix and non-text contrast remain unverified

Layout behaviour across phone, tablet and laptop widths, and the 3:1 contrast requirement for borders, focus indicators and chart colours, cannot be established by static analysis. These require either physical devices or an automated browser harness, neither of which is currently in place.

Operational readiness

F-19 • VERIFICATION GAP

MEDIUM

OPEN

Tenancy separation has not been demonstrated with two real identities

An automated matrix covers 63 tenant-scoped tables and a live rollback-only probe confirmed a non-administrative identity retrieves zero rows belonging to other companies or workspaces. That is database-layer evidence. It does not demonstrate that the signed-in interface honours the same boundary, nor that removing, reactivating or reassigning a worker takes effect immediately.

Owner action – a rehearsal using an owner account and a genuinely separate restricted-worker account, covering workspace and module visibility, the embedded task application, and access changes taking effect.

F-20 • COMMUNICATIONS

MEDIUM

OPEN

No mail path has been proven to reach a monitored inbox

Verification, invitation, password-recovery and support-report paths exist in code and the invitation flow includes a copy-link fallback. None has been confirmed to deliver to a real monitored mailbox, and sender domain and template configuration remain unconfirmed.

F-21 • ADOPTION

LOW

OPEN

The end-of-day reporting feature has never been used

`eod_reports` contains zero rows. The four crew members the feature was built for hold no accounts, so the workflow has not been exercised by its intended users and no feedback on its fitness exists. Discovered incidentally while probing an unrelated finding.

Owner action – supply four email addresses and issue invitations.

F-22 • MONITORING

LOW

PARTIALLY REMEDIATED

Browser-side failures were invisible; consolidated health reporting is still absent

Client-side error reporting is now live, capturing tenant and revision context, and was verified against production including confirmation that query strings are stripped from recorded URLs so tokens cannot be logged.

A consolidated view spanning browser, API, email and slow-route health does not exist. Defining what it should surface is a product decision rather than an engineering one.

Recommendations, in order

1 — Enable leaked-password protection (F-05)

A single dashboard setting, no engineering dependency, and the only open finding that closes with one action.

2 — Issue the four crew accounts (F-21)

The longest-standing item. Until the intended users exercise the end-of-day workflow there is no evidence it meets their needs, and the cost of discovering that later rises with every feature built on top of it.

3 — Conduct the two-identity rehearsal (F-19, F-20)

The remaining verification gap with the greatest consequence. Database-layer isolation is evidenced; interface-layer isolation is assumed. These should be exercised together, since the rehearsal requires a working invitation path.

4 — Schedule the bundle separation (F-13)

Not urgent for users, but it blocks further development and the cost of deferring rises as more code accumulates in the main bundle. Requires the ability to exercise the App Builder manually.

5 — Work through the handoff review (F-09)

Bounded, one-off, and safe to perform incrementally. New records are already protected.

Assessment

The system's tenancy boundary is materially stronger than at the baseline revision: every tenant-scoped table enforces row-level security with a readable policy, every foreign key is indexed, no policy re-evaluates authentication per row, and two genuine isolation defects have been closed and demonstrated closed against production.

The two most serious findings in this audit — F-01 and F-06 — shared a characteristic worth stating plainly. Neither appeared on the project's own release checklist, neither was detected by a test suite of nine hundred tests, and neither surfaced from advisor output as presented. Both were found by reading code with the specific question "what does this actually enforce?" A tautological comparison and an unconditional write both read as ordinary code and produce no signal until data is lost. Periodic close reading of authorisation and write paths should be treated as a standing practice rather than an activity that concludes.

The system is not ready for unrestricted public launch, and the reason is no longer principally engineering. Of the five open findings, four require a person or a decision rather than implementation. That is a materially better position than the baseline, where the outstanding work was largely unbuilt.